

Cost Function with Regularization

Regularization works by modifying the cost function to penalize large parameter values, which encourages the model to be simpler and less prone to overfitting.

The Core Idea: Penalizing Large Parameters

- **Problem:** In an overfit model (e.g., a high-order polynomial), the parameter values (w_j) are often very large, leading to a complex, "wiggly" function.
 - **Solution:** We can modify the cost function to not only minimize the error on the training data but also to keep the parameter values small.
 - **Example:** To simplify a 4th-order polynomial, we could add penalty terms like $+1000w_3^2 + 1000w_4^2$ to the cost function. To minimize this new cost, the algorithm is forced to choose very small values for w_3 and w_4 , effectively making the model behave more like a simpler quadratic function.
-

The Regularized Cost Function

Instead of penalizing specific parameters, regularization is typically applied to *all* weight parameters, w_j .

- The original cost function is modified by adding a **regularization term**:

$$J(\vec{w}, b) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2}_{\text{Original Cost}} + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{Regularization Term}}$$

- **Components:**
 - **Original Cost:** This part encourages the model to fit the training data well.
 - **Regularization Term:** This part encourages the parameters (w_j) to be small, which helps prevent overfitting.
 - **λ (Lambda):** The **regularization parameter**. This is a value you choose that controls the trade-off between the two goals.
 - **Convention:**
 - The bias parameter, b , is typically **not** regularized.
 - The term is scaled by $\frac{1}{2m}$ to make the math cleaner and help keep λ consistent even if the training set size changes.
-

The Role of the Regularization Parameter (λ)

The value of λ determines the strength of the regularization penalty and critically affects the model's final behavior.

- **If $\lambda = 0$:** The regularization term becomes zero. The model is not regularized at all and will likely **overfit**.
- **If λ is very large (e.g., 10^{10}):** The penalty for large parameters is huge. To minimize the cost, the algorithm will force all w_j parameters to be extremely close to zero. The model becomes too simple (e.g., $f(\vec{x}) \approx b$), resulting in a flat line that **underfits** the data.
- **If λ is "just right":** The model finds a good balance between fitting the data and keeping the parameters small, resulting in a model that **generalizes well**.